any changes to the data, and whether the changeset is valid or not. Start an IEx session to try it out:

```
$ iex -S mix
iex> alias PhoneApp.Conversations.Schema.Contact
iex> Contact.changeset(%{})
#Ecto.Changeset<
  action: nil,
  changes: %{},
  errors: [phone_number: {"can't be blank", [validation: :required]}],
  data: #PhoneApp.Conversations.Schema.Contact<>,
  valid?: false
>

iex> Contact.changeset(%{phone_number: "x"})
#Ecto.Changeset<
  action: nil,
  changes: %{phone_number: "x"},
  errors: [],
  data: #PhoneApp.Conversations.Schema.Contact<>,
  valid?: true
>
```

We could write a validation function to guarantee that the phone_number field is a valid E.164 formatted number, but that would be overkill for this project.

Try out the SmsMessage changeset as well. The update_changeset/2 function requires that we pass a %SmsMessage{} struct in—this would be the existing data we're updating:

```
iex> alias PhoneApp.Conversations.Schema.SmsMessage
iex> struct = %SmsMessage{status: "test"}
iex> SmsMessage.update_changeset(%{}, struct)
#Ecto.Changeset<action: nil, changes: %{}, errors: [],
 data: #PhoneApp.Conversations.Schema.SmsMessage<>, valid?: true>

iex> SmsMessage.update_changeset(%{status: ""}, struct)
#Ecto.Changeset<errors: [status: {"can't be blank", []}], valid?: false>
```

The last response is truncated, but the main point is that the combination of the input struct and the parameters determines the changeset validity. When we pass in a struct that has :status set, the changeset is valid. But when we pass in the same struct and try to clear the :status field, it's no longer valid.

We'll use these changesets to create our data layer next.